

Test your math and physics background, a reminder on Linear Algebra and Statistics

Morten Hjorth-Jensen¹

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway¹

January 22, 2025

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Test your self

The aim here is to present some basic mathematics (mostly linear algebra and complex numbers), statistics and quantum mechanics needed as a foundation for taking this course. You may not remember all the material you've gone through during your years of study, and some of the material may be new to you. Thus don't be intimidated if your memory fails you. We will revisit (with ditto exercises) several these topics during the semester..

Layout

The basic layout of the material covered by these slides is as follows:

1. A short repetition and definitions of basic concepts (e.g., eigenvalue problems, SVD, density matrices etc) and
2. Selected exercises after each topic

Vectors

We start by defining a vector $\mathbf{x}$ with n components, with x_0 as our first element, as

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix}.$$

and its transpose

$$\mathbf{x}^T = [x_0 \quad x_1 \quad x_2 \quad \dots \quad \dots \quad x_{n-1}],$$

In case we have a complex vector we define the hermitian conjugate

$$\mathbf{x}^\dagger = [x_0^* \quad x_1^* \quad x_2^* \quad \dots \quad \dots \quad x_{n-1}^*],$$

With a given vector $\mathbf{x}$, we define the inner product as

$$\mathbf{x}^T \mathbf{x} = \sum_{i=0}^{n-1} x_i x_i = x_0^2 + x_1^2 + \dots + x_{n-1}^2.$$

Hermitian conjugate

The hermitian conjugate of for example a matrix is obtained by taking the complex conjugate of each element and then taking the transpose of the resulting matrix. Often we will just say the transpose or just the conjugate, it should be clear from the context that we will mainly deal with hermitian quantities and our matrices will in most cases be square matrices.

Unitarity, as we will see below, plays also a central role in this course.

Outer products

In addition to inner products between vectors/states, the outer product plays a central role in many applications. It is defined as

$$\mathbf{xy}^T = \begin{bmatrix} x_0y_0 & x_0y_1 & x_0y_2 & \dots & \dots & x_0y_{n-2} & x_0y_{n-1} \\ x_1y_0 & x_1y_1 & x_1y_2 & \dots & \dots & x_1y_{n-2} & x_1y_{n-1} \\ x_2y_0 & x_2y_1 & x_2y_2 & \dots & \dots & x_2y_{n-2} & x_2y_{n-1} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ x_{n-2}y_0 & x_{n-2}y_1 & x_{n-2}y_2 & \dots & \dots & x_{n-2}y_{n-2} & x_{n-2}y_{n-1} \\ x_{n-1}y_0 & x_{n-1}y_1 & x_{n-1}y_2 & \dots & \dots & x_{n-1}y_{n-2} & x_{n-1}y_{n-1} \end{bmatrix}$$

The latter defines also our basic matrix layout.

Basic Matrix Features

A general $n \times n$ matrix is given by

$$\mathbf{A} = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & \dots & a_{0n-2} & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \dots & \dots & a_{1n-2} & a_{1n-1} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ a_{n-20} & a_{n-21} & a_{n-22} & \dots & \dots & a_{n-2n-2} & a_{n-2n-1} \\ a_{n-10} & a_{n-11} & a_{n-12} & \dots & \dots & a_{n-1n-2} & a_{n-1n-1} \end{bmatrix},$$

or in terms of its column vectors $\mathbf{a}_i$ as

$$\mathbf{A} = [\mathbf{a}_0 \quad \mathbf{a}_1 \quad \mathbf{a}_2 \quad \dots \quad \dots \quad \mathbf{a}_{n-2} \quad \mathbf{a}_{n-1}].$$

We can think of a matrix as a diagram of in general n rows and m columns. In the example here we have a square matrix.

The inverse of a matrix

The inverse of a square matrix (if it exists) is defined by

$$\mathbf{A}^{-1} \cdot \mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the unit matrix.

Selected Matrix Features

Relations	Name	matrix elements
$\mathbf{A} = \mathbf{A}^T$	symmetric	$a_{ij} = a_{ji}$
$\mathbf{A} = (\mathbf{A}^T)^{-1}$	real orthogonal	$\sum_k a_{ik} a_{jk} = \sum_k a_{ki} a_{kj} = \delta_{ij}$
$\mathbf{A} = \mathbf{A}^*$	real matrix	$a_{ij} = a_{ij}^*$
$\mathbf{A} = \mathbf{A}^\dagger$	hermitian	$a_{ij} = a_{ji}^*$
$\mathbf{A} = (\mathbf{A}^\dagger)^{-1}$	unitary	$\sum_k a_{ik} a_{jk}^* = \sum_k a_{ki}^* a_{kj} = \delta_{ij}$

Some famous Matrices

- ▶ Diagonal if $a_{ij} = 0$ for $i \neq j$
- ▶ Upper triangular if $a_{ij} = 0$ for $i > j$
- ▶ Lower triangular if $a_{ij} = 0$ for $i < j$
- ▶ Upper Hessenberg if $a_{ij} = 0$ for $i > j + 1$
- ▶ Lower Hessenberg if $a_{ij} = 0$ for $i < j - 1$
- ▶ Tridiagonal if $a_{ij} = 0$ for $|i - j| > 1$
- ▶ Lower banded with bandwidth p : $a_{ij} = 0$ for $i > j + p$
- ▶ Upper banded with bandwidth p : $a_{ij} = 0$ for $i < j - p$
- ▶ Banded, block upper triangular, block lower triangular....

Matrix Features

Some equivalent statements for square matrices

For an $n \times n$ matrix $\mathbf{A}$ the following properties are all equivalent

- ▶ If the inverse of $\mathbf{A}$ exists, $\mathbf{A}$ is nonsingular.
- ▶ The equation $\mathbf{Ax} = 0$ implies $\mathbf{x} = 0$.
- ▶ The rows of $\mathbf{A}$ form a basis of $\mathbb{C}^n$.
- ▶ The columns of $\mathbf{A}$ form a basis of $\mathbb{C}^n$.
- ▶ $\mathbf{A}$ is a product of elementary matrices. Can you name one example?
- ▶ 0 is not an eigenvalue of $\mathbf{A}$.

Important Mathematical Operations

The basic matrix operations that we will deal with are addition and subtraction

$$\mathbf{A} = \mathbf{B} \pm \mathbf{C} \implies a_{ij} = b_{ij} \pm c_{ij},$$

and scalar-matrix multiplication

$$\mathbf{A} = \gamma \mathbf{B} \implies a_{ij} = \gamma b_{ij}.$$

Vector-matrix and Matrix-matrix multiplication

We have also vector-matrix multiplications

$$\mathbf{y} = \mathbf{Ax} \implies y_i = \sum_{j=0}^{n-1} a_{ij}x_j,$$

and matrix-matrix multiplications

$$\mathbf{A} = \mathbf{BC} \implies a_{ij} = \sum_{k=0}^{n-1} b_{ik}c_{kj},$$

and transpositions of a matrix

$$\mathbf{A} = \mathbf{B}^T \implies a_{ij} = b_{ji}.$$

Important Mathematical Operations

Similarly, important vector operations that we will deal with are addition and subtraction

$$\mathbf{x} = \mathbf{y} \pm \mathbf{z} \implies x_i = y_i \pm z_i,$$

scalar-vector multiplication

$$\mathbf{x} = \gamma \mathbf{y} \implies x_i = \gamma y_i,$$

Other important mathematical operations

and vector-vector multiplication (called Hadamard multiplication)

$$\mathbf{x} = \mathbf{y}\mathbf{z} \implies x_i = y_i z_i.$$

Finally, as already mentioned, the inner or so-called dot product resulting in a constant

$$x = \mathbf{y}^T \mathbf{z} \implies x = \sum_{j=0}^{n-1} y_j z_j,$$

and the outer product, which yields a matrix,

$$\mathbf{A} = \mathbf{y}\mathbf{z}^T \implies a_{ij} = y_i z_j,$$

Defining basis states and quantum mechanical operators

We extend now to quantum mechanics our definitions of vectors, matrices and more.

We start by defining a state vector $\mathbf{x}$ (meant to represent various quantum mechanical degrees of freedom) with n components as

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix} .$$

Dirac notation

Throughout these notes we will use the so-called Dirac **bra-ket** formalism and we will replace the above standard boldfaced notation for a vector with

$$\mathbf{x} = |x\rangle = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix},$$

and

$$\mathbf{x}^\dagger = \langle x| = \begin{bmatrix} x_0^* & x_1^* & x_2^* & \dots & \dots & x_{n-1}^* \end{bmatrix},$$

Inner product in Dirac notation

With a given vector $|x\rangle$, we define the inner product as

$$\langle x|x\rangle = \sum_{i=0}^{n-1} x_i^* x_i = x_0^2 + x_1^2 + \cdots + x_{n-1}^2.$$

For two arbitrary vectors $|x\rangle$ and $|y\rangle$ with the same length, we have the general expression

$$\langle y|x\rangle = \sum_{i=0}^{n-1} y_i^* x_i = y_0^* x_0 + y_1^* x_1 + \cdots + y_{n-1}^* x_{n-1}.$$

The inner product is a real number

Note well that the inner product $\langle x|x \rangle$ is always a real number while for two different vectors $\langle y|x \rangle$ is in general not equal to $\langle x|y \rangle$, as can be seen from the example in the next slide.

We note in bypassing that $|x\rangle^\dagger = \langle x|$, $\langle x|^\dagger = |x\rangle$ and $(|x\rangle^\dagger)^\dagger = |x\rangle$.

Examples

Let us assume that $|x\rangle$ is given by

$$|x\rangle = \begin{bmatrix} 1 - i \\ 2 + i \end{bmatrix}.$$

The inner product gives us

$$\langle x|x\rangle = (1 + i)(1 - i) + (2 - i)(2 + i) = 7,$$

a real number.

Norm

We can use the norm/inner product to normalize the vector $|x\rangle$ and obtain

$$|x\rangle = \frac{1}{\sqrt{7}} \begin{bmatrix} 1 - i \\ 2 + i \end{bmatrix}.$$

As another example, consider the two vectors

$$|x\rangle = \begin{bmatrix} -1 \\ 2i \\ 1 \end{bmatrix},$$

and

$$|y\rangle = \begin{bmatrix} 1 \\ 0i \\ i \end{bmatrix}.$$

We see that the inner products $\langle x|y\rangle = -1 + i$, which is not the same as $\langle y|x\rangle = -1 - i$. This leads to the important rule

$$\langle x|y\rangle^* = \langle y|x\rangle.$$

Outer products

In addition to inner products between vectors/states, the outer product plays a central role in all of quantum mechanics. It is defined as

$$|x\rangle\langle y| = \begin{bmatrix} x_0 y_0^* & x_0 y_1^* & x_0 y_2^* & \dots & \dots & x_0 y_{n-2}^* & x_0 y_{n-1}^* \\ x_1 y_0^* & x_1 y_1^* & x_1 y_2^* & \dots & \dots & x_1 y_{n-2}^* & x_1 y_{n-1}^* \\ x_2 y_0^* & x_2 y_1^* & x_2 y_2^* & \dots & \dots & x_2 y_{n-2}^* & x_2 y_{n-1}^* \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ x_{n-2} y_0^* & x_{n-2} y_1^* & x_{n-2} y_2^* & \dots & \dots & x_{n-2} y_{n-2}^* & x_{n-2} y_{n-1}^* \\ x_{n-1} y_0^* & x_{n-1} y_1^* & x_{n-1} y_2^* & \dots & \dots & x_{n-1} y_{n-2}^* & x_{n-1} y_{n-1}^* \end{bmatrix}$$

Different operators and gates

In quantum computing, the so-called Pauli matrices, and other simple 2×2 matrices, play an important role, ranging from the setup of quantum gates to a rewrite of creation and annihilation operators and other quantum mechanical operators. Let us start with the familiar Pauli matrices and remind ourselves of some of their basic properties.

The Pauli matrices are defined as

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$\sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

and

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Properties of Pauli matrices

It is easy to show that the matrices obey the properties (being involutory)

$$\sigma_x \sigma_x = \sigma_y \sigma_y = \sigma_z \sigma_z = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

that is their products with themselves result in the identity matrix I . Furthermore, the Pauli matrices are unitary matrices meaning that their inverses are equal to their hermitian conjugated matrices. The determinants of the Pauli matrices are all equal to -1 , as can be easily verified.

Commutation relations

The Pauli matrices obey also the following commutation rules

$$[\sigma_x, \sigma_y] = 2i\sigma_z.$$

Before we proceed with other matrices and how they can be used to operate on various quantum mechanical states, let us try to define various basis sets and their pertinent notations. We will often refer to these basis states as our computational basis.

Definition of Computational basis states

Assume we have a two-level system where the two states are represented by the state vectors $|\phi_0\rangle$ and $|\phi_1\rangle$, respectively. These states could represent selected or effective degrees of freedom for either a single particle (fermion or boson) or they could represent effective many-body degrees of freedom. In actual realizations of quantum computing we search often for candidate systems where we can use some low-lying states as computational basis states. But we are not limited to quantum computing. When doing many-body physics, due to the exploding degrees of freedom, we normally search after effective ways by which we can reduce the involved dimensionalities to a number of degrees of freedom we can handle by a given many-body method.

See handwritten notes January 17 for more details

Projection operators

We will now relabel the above two states as two orthogonal and normalized basis (ONB) states

$$|\phi_0\rangle = |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and

$$|\phi_1\rangle = |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Identity and projection operators

It is straight forward to see that $\langle 1|0\rangle = 0$. With these two states we can define the identity operator I as the sum of the outer products of these two states, namely

$$I = \sum_{i=0}^{i=1} |i\rangle\langle i| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

We can further define the projection operators

$$P = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

and

$$Q = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Idempotent operators

We note that $P^2 = P$, $Q^2 = Q$ (the operators are idempotent) and that their determinants are zero, meaning in turn that we cannot use these operators for unitary/orthogonal transformations.

However, they play important roles in defining effective Hilbert spaces for many-body studies. Finally, before proceeding we note also that the two matrices commute and we have $\mathbf{PQ} = 0$ and $[\mathbf{P}, \mathbf{Q}] = 0$.

Superposition and more

Using the properties of ONBs we can expand a new state in terms of the above states. These states could also form a basis which is an eigenbasis of a selected Hamiltonian (more of this below).

We define now a new state which is a linear expansion in terms of these computational basis states

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where the coefficients $\alpha = \langle 0|\psi\rangle$ and $\beta = \langle 1|\psi\rangle$ represent the overlaps between the computational basis states and the state $|\psi\rangle$. In quantum speech, we say the state is in a superposition of the states $|0\rangle$ and $|1\rangle$.

Inner products

Computing the inner product of $|\psi\rangle$ we obtain

$$\langle\psi|\psi\rangle = |\alpha|^2\langle 0|0\rangle + |\beta|^2\langle 1|1\rangle = |\alpha|^2 + |\beta|^2 = 1,$$

since the new basis, which is defined in terms of a unitary/orthogonal transformation, preserves the orthogonality and norm of the original computational basis $|0\rangle$ and $|1\rangle$. To see this, consider the unitary transformation (show derivation of preserving orthogonality).

Acting with projection operators

If we now act with the projection operators $\mathbf{P}$ and $\mathbf{Q}$ on the state $|\psi\rangle$ we get

$$\mathbf{P}|\psi\rangle = |0\rangle\langle 0|(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle,$$

that is we **project** out the $|0\rangle$ component of the state $|\psi\rangle$ with the coefficient α while $\mathbf{Q}$ projects out the $|1\rangle$ component with coefficient β as seen from

$$\mathbf{Q}|\psi\rangle = |1\rangle\langle 1|(\alpha|0\rangle + \beta|1\rangle) = \beta|1\rangle.$$

The above results can easily be derived by multiplying the pertinent matrices with the vectors $|0\rangle$ and $|1\rangle$, respectively.

Density matrix

Using the above linear expansion we can now define the density matrix of the state $|\psi\rangle$ as the outer product

$$\rho = |\psi\rangle\langle\psi| = \alpha\alpha^*|0\rangle\langle 0| + \alpha\beta^*|0\rangle\langle 1| + \beta\alpha^*|1\rangle\langle 0| + \beta\beta^*|1\rangle\langle 1|,$$

which leads to

$$\rho = \begin{bmatrix} \alpha\alpha^* & \alpha\beta^* \\ \beta\alpha^* & \beta\beta^* \end{bmatrix}.$$

Finally, we note that the trace of the density matrix is simply given by unity

$$\text{tr}\rho = \alpha\alpha^* + \beta\beta^* = 1.$$

Other important matrices

We present other operators (as matrices) which play an important role in quantum computing, the so-called Hadamard matrix (or gate as we will use later)

$$\mathbf{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The action of the operator $\mathbf{H}$ on a computational basis state like $|0\rangle$ gives

$$\mathbf{H}|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle),$$

and

$$\mathbf{H}|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

that is we create a superposition of the states $|0\rangle$ and $|1\rangle$.

Phase matrix

Another famous operation is the phase matrix given by

$$\mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}.$$

Tensor products

Consider now two vectors with length $n = 2$, with elements

$$|x\rangle = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix},$$

and

$$|y\rangle = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}.$$

The tensor product of these two vectors is defined as

$$|x\rangle \otimes |y\rangle = |xy\rangle = \begin{bmatrix} x_0 y_0 \\ x_0 y_1 \\ x_1 y_0 \\ x_1 y_1 \end{bmatrix},$$

which is now a vector of length 4.

Examples of tensor products

If we now go back to our original one-qubit basis states, we can form the following tensor products

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |00\rangle,$$

$$|0\rangle \otimes |1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle.$$

More states

$$|1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle,$$

and finally

$$|1\rangle \otimes |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle.$$

Three qubits

We have now four different states and we could try to make a new list by relabeling the states as follows $|00\rangle = |0\rangle$, $|01\rangle = |1\rangle$, $|10\rangle = |2\rangle$, $|11\rangle = |3\rangle$.

In similar ways we can define the tensor product of three qubits (or single-particle states) as

$$|0\rangle \otimes |0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |000\rangle,$$

which is a new vector of length eight.

Generalizing

We note that with a single-particle basis given the states $|0\rangle$ and $|1\rangle$ we can, with N particles construct 2^N different states. This is something we can generalize to

- ▶ discuss ways of labeling states
- ▶ how to write a code which does it

Tensor products of matrices

The tensor product of two 2×2 matrices $\mathbf{A}$ and $\mathbf{B}$ is given by

$$\mathbf{A} \times \mathbf{B} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \otimes \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} = \begin{bmatrix} a_{00}b_{00} & a_{00}b_{01} & a_{01}b_{00} & a_{01}b_{01} \\ a_{00}b_{10} & a_{00}b_{11} & a_{01}b_{10} & a_{01}b_{11} \\ a_{10}b_{00} & a_{10}b_{01} & a_{11}b_{00} & a_{11}b_{01} \\ a_{10}b_{10} & a_{10}b_{11} & a_{11}b_{10} & a_{11}b_{11} \end{bmatrix}$$

Measurements

The probability of a measurement on a quantum system giving a certain result is determined by the weight of the relevant basis state in the state vector. After the measurement, the system is in a state that corresponds to the result of the measurement. The operators and gates discussed below are examples of operations we can perform on specific states.

We consider the state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Definitions of measurements

1. A measurement can yield only one of the above states, either $|0\rangle$ or $|1\rangle$.
2. The probability of a measurement resulting in $|0\rangle$ is $\alpha^* \alpha = |\alpha|^2$.
3. The probability of a measurement resulting in $|1\rangle$ is $\beta^* \beta = |\beta|^2$.
4. And we note that the sum of the outcomes gives $\alpha^* \alpha + \beta^* \beta = 1$ since the two states are normalized.

After the measurement, the state of the system is the state associated with the result of the measurement.

We have already encountered the projection operators P and Q . Let us now look at other types of operations we can make on qubit states.

Different operators and gates

In quantum computing, the so-called Pauli matrices, and other simple 2×2 matrices, play an important role, ranging from the setup of quantum gates to a rewrite of creation and annihilation operators and other quantum mechanical operators. Let us start with the familiar Pauli matrices and remind ourselves of some of their basic properties.

Assume we operate with σ_x on our basis state $|0\rangle$. This gives

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

that is we switch from $|0\rangle$ to $|1\rangle$ (often called a spin flip operation) and similarly we have

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

More on Pauli matrices

This matrix plays an important role in quantum computing since we can link this with the classical **NOT** operation. If we send in bit 0, the **NOT** gate outputs bit 1 and vice versa. We can use the σ_x matrix to implement the quantum mechanical equivalent of a classical **NOT** gate. If we input what we could represent as bit 0 in terms of the basis state $|0\rangle$, operating on this state results in the state $|1\rangle$, which we in turn can interpret as the classical bit 1.

Linear superposition

If we have a linear superposition of these states we obtain

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}.$$

The σ_y matrix introduces an imaginary sign, which we will later encounter in terms of so-called phase-shift operations.

The σ_z matrix

The σ_z matrix has the following effect

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$

which we will also link with a specific phase-shift.

Unitarity

The matrices we introduced here are so-called unitary matrices. This is an important element in quantum mechanics since the evolution of a closed quantum system is described by operations involving unitary operations only.

We have defined a new state $|\psi_p\rangle$ as a linear expansion in terms of an orthogonal and normalized basis (our computational basis) ϕ_λ

$$|\psi_i\rangle = \sum_j u_{ij} |\phi_j\rangle. \quad (1)$$

Hamiltonians and basis functions

It is normal to choose a basis defined as the eigenfunctions of parts of the full Hamiltonian. The typical situation consists of the solutions of the one-body part of the Hamiltonian, that is we have

$$\hat{h}_0|\phi_i\rangle = \epsilon_i|\phi_i\rangle.$$

This is normally referred to as a single-particle basis $|\phi_i(\mathbf{r})\rangle$, defined by the quantum numbers i and $\mathbf{r}$.

Unitary transformations

A unitary transformation is important since it keeps the orthogonality. To see this consider first a basis of vectors $\mathbf{v}_i$,

$$\mathbf{v}_i = \begin{bmatrix} v_{i1} \\ \vdots \\ \vdots \\ v_{in} \end{bmatrix}$$

We assume that the basis is orthogonal, that is

$$\mathbf{v}_j^T \mathbf{v}_i = \delta_{ij}.$$

An orthogonal or unitary transformation

$$\mathbf{w}_i = \mathbf{U} \mathbf{v}_i,$$

preserves the dot product and orthogonality since

$$\mathbf{w}_j^T \mathbf{w}_i = (\mathbf{U} \mathbf{v}_j)^T \mathbf{U} \mathbf{v}_i = \mathbf{v}_j^T \mathbf{U}^T \mathbf{U} \mathbf{v}_i = \mathbf{v}_j^T \mathbf{v}_i = \delta_{ij}.$$

Orthogonality preserved

This means that if the coefficients $u_{p\lambda}$ belong to a unitary or orthogonal transformation (using the Dirac bra-ket notation)

$$|\psi_i\rangle = \sum_j u_{ij} |\phi_j\rangle.$$

orthogonality is preserved.

This property is extremely useful when we build up a basis of many-body determinant based states.

Note also that although a basis $\{|\phi_i\rangle\}$ contains an infinity of states, for practical calculations we have always to make some truncations.

Example

Assume we have two one-qubit states represented by

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix},$$

and

$$|\phi\rangle = \gamma|0\rangle + \delta|1\rangle = \begin{bmatrix} \gamma \\ \delta \end{bmatrix}.$$

We assume that the state $|\phi\rangle$ is obtained through a unitary transformation of $|\psi\rangle$ through a matrix $\mathbf{U}$ with its hermitian conjugate $\mathbf{U}^\dagger$ with matrix elements $u_{ij}^\dagger = u_{ji}^*$ and $\mathbf{I} = \mathbf{U}\mathbf{U}^\dagger = \mathbf{U}^\dagger\mathbf{U}$.

Inverse of unitary matrices

Note that this means that the hermitian conjugate of a unitary matrix is equal to its inverse. This has important consequences for what is called reversibility. We say quantum mechanics is a theory which is reversible with a probabilistic determinism. Classical mechanics on the other is reversible in a deterministic way, that is, knowing all initial conditions we can in principle determine the future motion of an object which obey the laws of motion of classical mechanics.

We have then

$$\begin{bmatrix} \gamma \\ \delta \end{bmatrix} = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}.$$

New basis is also orthogonal

Since our original basis $|\psi\rangle$ is orthogonal and normalized with $|\alpha|^2 + |\beta|^2 = 1$, the new basis is also orthogonal and normalized, as we can see below here.

Since the inverse of a hermitian matrix is equal to its hermitian conjugate/adjoint), unitary transformations are always reversible. Why are only unitary transformations allowed? The key lies in the way the inner product transforms.

To see this we rewrite the new basis from the previous example in its two components as

$$|\phi\rangle_i = \sum_j u_{ij} |\psi\rangle_j,$$

or in terms of a matrix-vector notation we have

$$|\phi\rangle = \mathbf{U}|\psi\rangle,$$

More on orthogonality

We have already assumed that $\langle \psi | \psi \rangle = |\alpha|^2 + |\beta|^2 = 1$.

We have that

$$\langle \phi | i = \sum_j u_{ij}^* \langle \psi | j,$$

or in terms of a matrix-vector notation we have

$$\langle \phi | = \langle \psi | \mathbf{U}^\dagger.$$

Note that the two vectors are row vectors now.

If we stay with this notation we have

$$\langle \phi | \phi \rangle = \langle \psi | \mathbf{U}^\dagger \mathbf{U} | \psi \rangle = \langle \psi | \psi \rangle = 1!$$

Unitary transformations are rotations in state space which preserve the length (the square root of the inner product) of the state vector.

Exercises to test yourself

To be added

Important Matrix and vector handling packages

There are several central software packages for linear algebra and eigenvalue problems. Several of the more popular ones have been wrapped into other software packages like those from the widely used text **Numerical Recipes**. The original source codes in many of the available packages are often taken from the widely used software package LAPACK, which follows two other popular packages developed in the 1970s, namely EISPACK and LINPACK. We describe them shortly here.

- ▶ LINPACK: package for linear equations and least square problems.
- ▶ LAPACK: package for solving symmetric, unsymmetric and generalized eigenvalue problems. From LAPACK's website <http://www.netlib.org> it is possible to download for free all source codes from this library. Both C/C++ and Fortran versions are available.
- ▶ BLAS (I, II and III): (Basic Linear Algebra Subprograms) are routines that provide standard building blocks for performing basic vector and matrix operations. Blas I is vector operations, II vector matrix operations and III matrix matrix operations.

Basic Matrix Features

Matrix properties reminder

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \quad I = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Basic Matrix Features

The inverse of a matrix is defined by

$$A^{-1} \cdot A = I$$

Basic Matrix Features

Matrix Properties Reminder

Relations	Name	matrix elements
$A = A^T$	symmetric	$a_{ij} = a_{ji}$
$A = (A^T)^{-1}$	real orthogonal	$\sum_k a_{ik} a_{jk} = \sum_k a_{ki} a_{kj} = \delta_{ij}$
$A = A^*$	real matrix	$a_{ij} = a_{ij}^*$
$A = A^\dagger$	hermitian	$a_{ij} = a_{ji}^*$
$A = (A^\dagger)^{-1}$	unitary	$\sum_k a_{ik} a_{jk}^* = \sum_k a_{ki}^* a_{kj} = \delta_{ij}$

Some famous Matrices

- ▶ Diagonal if $a_{ij} = 0$ for $i \neq j$
- ▶ Upper triangular if $a_{ij} = 0$ for $i > j$
- ▶ Lower triangular if $a_{ij} = 0$ for $i < j$
- ▶ Upper Hessenberg if $a_{ij} = 0$ for $i > j + 1$
- ▶ Lower Hessenberg if $a_{ij} = 0$ for $i < j - 1$
- ▶ Tridiagonal if $a_{ij} = 0$ for $|i - j| > 1$
- ▶ Lower banded with bandwidth p : $a_{ij} = 0$ for $i > j + p$
- ▶ Upper banded with bandwidth p : $a_{ij} = 0$ for $i < j - p$
- ▶ Banded, block upper triangular, block lower triangular....

Basic Matrix Features

Some Equivalent Statements

For an $N \times N$ matrix A the following properties are all equivalent

- ▶ If the inverse of A exists, A is nonsingular.
- ▶ The equation $Ax = 0$ implies $x = 0$.
- ▶ The rows of A form a basis of R^N .
- ▶ The columns of A form a basis of R^N .
- ▶ A is a product of elementary matrices.
- ▶ 0 is not eigenvalue of A .

Numpy and arrays

Numpy provides an easy way to handle arrays in Python. The standard way to import this library is as

```
import numpy as np
n = 10
x = np.random.normal(size=n)
print(x)
```

Here we have defined a vector x with $n = 10$ elements with its values given by the Normal distribution $N(0, 1)$. Another alternative is to declare a vector as follows

```
import numpy as np
x = np.array([1, 2, 3])
print(x)
```

Here we have defined a vector with three elements, with $x_0 = 1$, $x_1 = 2$ and $x_2 = 3$. Note that both Python and C++ start numbering array elements from 0 and on. This means that a vector with n elements has a sequence of entities $x_0, x_1, x_2, \dots, x_{n-1}$. We could also let (recommended) Numpy to compute the logarithms of a specific array as

```
import numpy as np
x = np.log(np.array([4, 7, 8]))
```

Matrices in Python

Having defined vectors, we are now ready to try out matrices. We can define a 3×3 real matrix $\hat{A}$ as (recall that we use lowercase letters for vectors and uppercase letters for matrices)

```
import numpy as np
A = np.log(np.array([ [4.0, 7.0, 8.0], [3.0, 10.0, 11.0], [4.0, 5.0, 7.0]
print(A)
```

If we use the **shape** function we would get (3,3) as output, that is verifying that our matrix is a 3×3 matrix. We can slice the matrix and print for example the first column (Python organized matrix elements in a row-major order, see below) as

```
import numpy as np
A = np.log(np.array([ [4.0, 7.0, 8.0], [3.0, 10.0, 11.0], [4.0, 5.0, 7.0]
# print the first column, row-major order and elements start with 0
print(A[:,0])
```

We can continue this was by printing out other columns or rows.

The example here prints out the second column

```
import numpy as np
A = np.log(np.array([ [4.0, 7.0, 8.0], [3.0, 10.0, 11.0], [4.0, 5.0, 7.0]
# print the first column, row-major order and elements start with 0
print(A[1,:])
```


Gaussian Elimination

We start with the linear set of equations

$$Ax = w.$$

We assume also that the matrix A is non-singular and that the matrix elements along the diagonal satisfy $a_{ii} \neq 0$. Simple 4×4 example

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}.$$

Gaussian Elimination

or

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = w_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = w_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = w_3$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = w_4.$$

Gaussian Elimination

The basic idea of Gaussian elimination is to use the first equation to eliminate the first unknown x_1 from the remaining $n - 1$ equations. Then we use the new second equation to eliminate the second unknown x_2 from the remaining $n - 2$ equations. With $n - 1$ such eliminations we obtain a so-called upper triangular set of equations of the form

$$b_{11}x_1 + b_{12}x_2 + b_{13}x_3 + b_{14}x_4 = y_1$$

$$b_{22}x_2 + b_{23}x_3 + b_{24}x_4 = y_2$$

$$b_{33}x_3 + b_{34}x_4 = y_3$$

$$b_{44}x_4 = y_4.$$

We can solve this system of equations recursively starting from x_n (in our case x_4) and proceed with what is called a backward substitution.

Gaussian Elimination

This process can be expressed mathematically as

$$x_m = \frac{1}{b_{mm}} \left(y_m - \sum_{k=m+1}^n b_{mk} x_k \right) \quad m = n-1, n-2, \dots, 1. \quad (2)$$

To arrive at such an upper triangular system of equations, we start by eliminating the unknown x_1 for $j = 2, n$. We achieve this by multiplying the first equation by a_{j1}/a_{11} and then subtract the result from the j th equation. We assume obviously that $a_{11} \neq 0$ and that A is not singular.

Gaussian Elimination

Our actual 4×4 example reads after the first operation

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & (a_{22} - \frac{a_{21}a_{12}}{a_{11}}) & (a_{23} - \frac{a_{21}a_{13}}{a_{11}}) & (a_{24} - \frac{a_{21}a_{14}}{a_{11}}) \\ 0 & (a_{32} - \frac{a_{31}a_{12}}{a_{11}}) & (a_{33} - \frac{a_{31}a_{13}}{a_{11}}) & (a_{34} - \frac{a_{31}a_{14}}{a_{11}}) \\ 0 & (a_{42} - \frac{a_{41}a_{12}}{a_{11}}) & (a_{43} - \frac{a_{41}a_{13}}{a_{11}}) & (a_{44} - \frac{a_{41}a_{14}}{a_{11}}) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ w_2^{(2)} \\ w_3^{(2)} \\ w_4^{(2)} \end{bmatrix},$$

or

$$\begin{aligned} b_{11}x_1 + b_{12}x_2 + b_{13}x_3 + b_{14}x_4 &= y_1 \\ a_{22}^{(2)}x_2 + a_{23}^{(2)}x_3 + a_{24}^{(2)}x_4 &= w_2^{(2)} \\ a_{32}^{(2)}x_2 + a_{33}^{(2)}x_3 + a_{34}^{(2)}x_4 &= w_3^{(2)} \\ a_{42}^{(2)}x_2 + a_{43}^{(2)}x_3 + a_{44}^{(2)}x_4 &= w_4^{(2)}, \end{aligned}$$

(3)

Gaussian Elimination

The new coefficients are

$$b_{1k} = a_{1k}^{(1)} \quad k = 1, \dots, n, \quad (4)$$

where each $a_{1k}^{(1)}$ is equal to the original a_{1k} element. The other coefficients are

$$a_{jk}^{(2)} = a_{jk}^{(1)} - \frac{a_{j1}^{(1)} a_{1k}^{(1)}}{a_{11}^{(1)}} \quad j, k = 2, \dots, n, \quad (5)$$

with a new right-hand side given by

$$y_1 = w_1^{(1)}, \quad w_j^{(2)} = w_j^{(1)} - \frac{a_{j1}^{(1)} w_1^{(1)}}{a_{11}^{(1)}} \quad j = 2, \dots, n. \quad (6)$$

We have also set $w_1^{(1)} = w_1$, the original vector element. We see that the system of unknowns $x_1, \dots, x_n$ is transformed into an $(n-1) \times (n-1)$ problem.

Gaussian Elimination

This step is called forward substitution. Proceeding with these substitutions, we obtain the general expressions for the new coefficients

$$a_{jk}^{(m+1)} = a_{jk}^{(m)} - \frac{a_{jm}^{(m)} a_{mk}^{(m)}}{a_{mm}^{(m)}} \quad j, k = m+1, \dots, n, \quad (7)$$

with $m = 1, \dots, n-1$ and a right-hand side given by

$$w_j^{(m+1)} = w_j^{(m)} - \frac{a_{jm}^{(m)} w_m^{(m)}}{a_{mm}^{(m)}} \quad j = m+1, \dots, n. \quad (8)$$

This set of $n-1$ eliminations leads us to an equations which is solved by back substitution. If the arithmetics is exact and the matrix A is not singular, then the computed answer will be exact. Even though the matrix elements along the diagonal are not zero, numerically small numbers may appear and subsequent divisions may lead to large numbers, which, if added to a small number may yield losses of precision. Suppose for example that our first division in $(a_{22} - a_{21}a_{12}/a_{11})$ results in -10^{-7} and that a_{22} is one. one. We

Linear Algebra Methods

- ▶ Gaussian elimination, $O(2/3n^3)$ flops, general matrix
- ▶ LU decomposition, upper triangular and lower tridiagonal matrices, $O(2/3n^3)$ flops, general matrix. Get easily the inverse, determinant and can solve linear equations with back-substitution only, $O(n^2)$ flops
- ▶ Cholesky decomposition. Real symmetric or hermitian positive definite matrix, $O(1/3n^3)$ flops.
- ▶ Tridiagonal linear systems, important for differential equations. Normally positive definite and non-singular. $O(8n)$ flops for symmetric. Special case of banded matrices.
- ▶ Singular value decomposition
- ▶ the QR method will be discussed in chapter 7 in connection with eigenvalue systems. $O(4/3n^3)$ flops.

LU Decomposition

The LU decomposition method means that we can rewrite this matrix as the product of two matrices L and U where

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix}.$$

LU Decomposition

LU decomposition forms the backbone of other algorithms in linear algebra, such as the solution of linear equations given by

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = w_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = w_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = w_3$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = w_4.$$

The above set of equations is conveniently solved by using LU decomposition as an intermediate step.

The matrix $A \in \mathbb{R}^{n \times n}$ has an LU factorization if the determinant is different from zero. If the LU factorization exists and A is non-singular, then the LU factorization is unique and the determinant is given by

$$\det\{A\} = \det\{LU\} = \det\{L\}\det\{U\} = u_{11}u_{22} \dots u_{nn}.$$

LU Decomposition, why?

There are at least three main advantages with LU decomposition compared with standard Gaussian elimination:

- ▶ It is straightforward to compute the determinant of a matrix
- ▶ If we have to solve sets of linear equations with the same matrix but with different vectors y , the number of FLOPS is of the order n^3 .
- ▶ The inverse is such an operation

LU Decomposition, linear equations

With the LU decomposition it is rather simple to solve a system of linear equations

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = w_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = w_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = w_3$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = w_4.$$

This can be written in matrix form as

$$Ax = w.$$

where A and w are known and we have to solve for x . Using the LU decomposition we write

$$Ax \equiv LUx = w.$$

LU Decomposition, linear equations

The previous equation can be calculated in two steps

$$Ly = w; \quad Ux = y.$$

To show that this is correct we use the LU decomposition to rewrite our system of linear equations as

$$LUx = w,$$

and since the determinant of L is equal to 1 (by construction since the diagonals of L equal 1) we can use the inverse of L to obtain

$$Ux = L^{-1}w = y,$$

which yields the intermediate step

$$L^{-1}w = y$$

and as soon as we have y we can obtain x through $Ux = y$.

LU Decomposition, why?

For our four-dimensional example this takes the form

$$y_1 = w_1$$

$$l_{21}y_1 + y_2 = w_2$$

$$l_{31}y_1 + l_{32}y_2 + y_3 = w_3$$

$$l_{41}y_1 + l_{42}y_2 + l_{43}y_3 + y_4 = w_4.$$

and

$$u_{11}x_1 + u_{12}x_2 + u_{13}x_3 + u_{14}x_4 = y_1$$

$$u_{22}x_2 + u_{23}x_3 + u_{24}x_4 = y_2$$

$$u_{33}x_3 + u_{34}x_4 = y_3$$

$$u_{44}x_4 = y_4$$

This example shows the basis for the algorithm needed to solve the set of n linear equations.

LU Decomposition, linear equations

The algorithm goes as follows

- ▶ Set up the matrix A and the vector w with their correct dimensions. This determines the dimensionality of the unknown vector x .
- ▶ Then LU decompose the matrix A through a call to the function `ludcmp(double a, int n, int indx, double &d)`. This function returns the LU decomposed matrix A , its determinant and the vector `indx` which keeps track of the number of interchanges of rows. If the determinant is zero, the solution is malconditioned.
- ▶ Thereafter you call the function `lubksb(double a, int n, int indx, double w)` which uses the LU decomposed matrix A and the vector w and returns x in the same place as w . Upon exit the original content in w is destroyed. If you wish to keep this information, you should make a backup of it in your calling function.

LU Decomposition, the inverse of a matrix

If the inverse exists then

$$A^{-1}A = I,$$

the identity matrix. With an LU decomposed matrix we can rewrite the last equation as

$$LUA^{-1} = I.$$

LU Decomposition, the inverse of a matrix

If we assume that the first column (that is column 1) of the inverse matrix can be written as a vector with unknown entries

$$A_1^{-1} = \begin{bmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ \dots \\ a_{n1}^{-1} \end{bmatrix},$$

then we have a linear set of equations

$$LU \begin{bmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ \dots \\ a_{n1}^{-1} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}.$$

LU Decomposition, the inverse

In a similar way we can compute the unknown entries of the second column,

$$\text{LU} \begin{bmatrix} a_{12}^{-1} \\ a_{22}^{-1} \\ \dots \\ a_{n2}^{-1} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ \dots \\ 0 \end{bmatrix},$$

and continue till we have solved all n sets of linear equations.